

Student Resource 3 Documentation

Version 1 Documentation

Why do feature selection at all?

At equal performance, a simpler model is better. If a model with 5 features scores the same as a model with 13, the 5-feature model wins. It is easier to explain to a stakeholder, easier to maintain, less likely to break when data changes, and less prone to memorising noise

Faster. Fewer columns means less data to process during training and prediction. For a small dataset, you will barely notice. For millions of rows and hundreds of columns, it is the difference between a model that trains in minutes and one that trains in hours.

Improve the model's performance. Irrelevant or redundant features add noise. A model can latch onto random patterns in those columns, which hurts it on new data. Removing them can raise accuracy or F1 score on unseen data, not just speed.

Do it before hyperparameter tuning and model selection. This is about order of operations. If you tune a model on a messy feature set and then change the features, the tuning is wasted because the best hyperparameters depend on the inputs. So the sensible pipeline is: clean data, select features, then tune and compare models.

Setup and the baseline model

Version 1: import pandas, NumPy and matplotlib, load the wine dataset from scikit-learn (load_wine), and turn on inline plotting with %matplotlib inline.

One small thing I caught while replicating: the notes have import pandas as np. That is a typo. NumPy is the library that normally takes the np alias, so the correct line is import numpy as np. Using the wrong import would break any later np. call, so I treated it as a correction and not as something to copy.

The baseline is a **Gradient Boosting Classifier trained on all features**, scored with **F1**. This step matters more than it looks. You cannot say a feature selection method helped unless you know what the model scored without it. The baseline is the benchmark, and every technique afterwards gets compared against it. Gradient boosting builds many small decision trees one after another, each trying to fix the mistakes of the previous ones, and it can also report how important each feature was, which becomes useful for RFE and Boruta.

F1 score is the balance between precision (of the things predicted positive, how many really were) and recall (of the things that really were positive, how many did we catch). It is a better

single number than plain accuracy when classes are uneven, and for a multi-class dataset like wine, you use an averaged version.

The techniques in Version 1

The notes list five items: Variance Threshold, K Best Features, Recursive Feature Elimination (RFE), and Boruta. (The outline at the top mentions RFE twice, once spelled out and once abbreviated, so there are really four distinct methods.)

Variance Threshold

- **What it is:** A very simple filter. It looks at each feature on its own and removes the ones that barely change.
- **How it works:** Variance measures how spread out values are around their mean. If a feature has a variance of 0, every row has the same value, so it is a constant. You pick a threshold and drop any feature whose variance falls below it. The default threshold of 0 removes only constants.
- **Why it matters:** A column that never changes cannot help a model tell one row from another. It is the cheapest cleanup you can do and a sensible first pass.

K Best Features (filter method)

- **What it is:** Score every feature using some statistical measure of importance against the target, then keep the top k.
- **How it works:** You choose a scoring function (for example, chi-squared or ANOVA F-value in scikit-learn's SelectKBest) and a value for k. Each feature gets a score, they are ranked, and the best k survive.
- **Why it matters:** It is fast because no model has to be trained to do the scoring. It also gives you a ranked list you can inspect.

Recursive Feature Elimination (RFE)

- **What it is:** A wrapper method. It uses an actual model to decide which features to throw away.
- **How it works:** Train the model on all features, rank them by importance, remove the weakest one (or few), retrain, and repeat until the desired number remains.
- **Why it matters:** Because a real model is judging the features, it takes interactions between features into account in a way simple filters cannot.
- **Key condition from the notes:** The model must have a way of calculating feature importance. Tree-based models work (they give importances), and so do linear models (they give coefficients). A model that exposes neither cannot drive RFE.
-

Boruta

Boruta is the method that "takes the human out". The point is that you do not set a subjective cutoff like k or a variance threshold; the algorithm builds its own benchmark from the data. It works in three steps:

1. **Create shadow features.** For every real feature, make a copy and randomly shuffle its values. Shuffling destroys any real relationship with the target, so these shadow columns are pure noise by construction.
2. **Compute importance for everything.** Train a model (tree-based, such as a random forest or gradient boosting) on real and shadow features together. Take the highest importance score among the shadow features. That becomes the threshold.
3. **Compare.** If a real feature's importance is higher than that threshold, keep it. Otherwise, discard it.

The reasoning is neat. If a real feature cannot even beat the best random-noise column, it is not showing any genuine signal.

Worked example from the resource

Column	Importance (I)	Decision
Feature 1	4	Discard (below 10)
Feature 2	12	Keep (above 10)
Feature 3	11	Keep (above 10)
Shadow_1	2	Noise
Shadow_2	10	Noise, and the highest, so this is the threshold
Shadow_3	5	Noise

Key takeaways from Version 1

- Feature selection is justified by simplicity, speed, and sometimes better performance, and it belongs before tuning.
- Always build a baseline with all features first; otherwise, there is nothing to compare against.
- The methods run from cheap and crude (variance threshold) to expensive and smarter (RFE, Boruta).
- Variance threshold and K best need you to pick a number. Boruta avoids that by using shuffled copies as its own yardstick.
- RFE and Boruta both depend on a model that can report feature importance.

Version 2 Documentation

Need for feature selection

With every feature thrown in, a model reaches only around 60% accuracy, but with relevant and well-chosen features, it performs around 80%. More features do not mean a better model. Irrelevant inputs dilute the useful ones.

What is feature selection?

- Selecting the optimal and best features for the machine learning model.
- Removing irrelevant or partially relevant features from the data.

Importance of feature selection

Benefit	What it means in plain terms
Reduces the curse of dimensionality	As the number of features grows, data becomes sparse in that space and models need far more rows to learn reliably. Fewer features eases this.
Helps in learning the model	A cleaner input makes it easier for the algorithm to find the real pattern.
Minimizes the cost of computation	Less memory and less training time.
Helps in achieving good accuracy	Less noise usually means better predictions on new data.

These overlap strongly with Version 1's four reasons (simpler, faster, better performance)

The classification of techniques

We have **supervised** and **unsupervised**. Supervised methods use the target variable to judge features; unsupervised methods look only at the features themselves (for example, variance or correlation between features). The supervised branch then divides into three families: **filter**, **wrapper**, and **embedded** methods. This is the framework that organises everything else in the deck.

Filter methods

Filter methods score features using statistics before any model is trained. They are independent of the classifier.

- **Information Gain:** Measures how much the uncertainty (entropy) in the target drops once you know the feature's value. A bigger drop means a more informative feature.
- **Chi-Squared test:** Tests the relationship between categorical variables; in other words, whether a feature and the target are independent of each other. A higher score suggests a stronger relationship. In scikit-learn, chi2 needs non-negative feature values.
- **Fisher's Score:** Ranks features in descending order of how well they separate the classes.

Wrapper methods

Wrapper methods "wrap" a model around the selection process. They try different feature subsets, train the model each time, and judge subsets by performance.

- **Forward Selection:** Start with no features, add the one that helps most, repeat, and evaluate at each step.
- **Backward Elimination:** Start with every feature, and at each step remove the least significant one.
- **Exhaustive Feature Selection:** Tries every possible combination (brute force). It finds the best subset but becomes impractical very quickly as features multiply.
- **Recursive Feature Elimination:** A recursive, greedy optimisation approach. It repeatedly fits a model and prunes the weakest features.

Embedded methods

Embedded methods perform selection as part of training the model itself.

- **Regularisation (L1, L2, Elastic Net):** A penalty on the size of coefficients is added to the loss. L1 (Lasso) can shrink some coefficients to exactly zero, which removes those features. L2 (Ridge) shrinks coefficients toward zero without removing them, so it reduces the influence of weak features. Elastic Net mixes both.
- **Random Forest and tree-based methods:** Trees naturally produce feature importance scores, so selection falls out of the model.

Advantages and disadvantages

	Filter method	Wrapper method
Advantages	Better computational complexity; independent of classifier; fast and scalable	Interacts with the classifier; model is feature dependent; minimizes computational cost (as worded on the slide)

Disadvantages	Ignores interaction with classifier; ignores feature dependencies; vagueness in filter method	Computationally intensive; high risk of overfitting; lower discriminative power
----------------------	---	---

The hands-on work

All the code in Version 2 uses the **Pima Indians diabetes dataset**.

Loading and preparing the data

```
import pandas as pd
import numpy as np

data =
"https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"
features = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
df = pd.read_csv(data, names=features)

df.shape      # (768, 9)

data = df.values
X = data[:, 0:8]      # the 8 input features
Y = data[:, 8] # the target (class)
```

Filter method: SelectKBest with chi-squared

```
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2

chi_best = SelectKBest(score_func=chi2, k=4)
k_best = chi_best.fit(X, Y)

np.set_printoptions(precision=3)
print(k_best.scores_)

k_features = k_best.transform(X)
print(k_features[0:5, :])
```

Here, SelectKBest is told to score with chi-squared and keep the top 4 features. fit calculates a score for each of the 8 features, scores_ lets you see them, and transform returns a new array containing only the four winning columns. Printing the first five rows is a quick sanity check that you really did end up with four columns. Chi-squared works here because all of the values are non-negative. (The slide images I worked from were blurry, so k=4 and precision=3 are my best reading of the code.)

Wrapper method: Recursive Feature Elimination

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

import warnings
warnings.filterwarnings('ignore')

model_lr = LogisticRegression()
recur_fe = RFE(model_lr, 3)
Feature = recur_fe.fit(X, Y)
print("Number of Features: %s" % (Feature.n_features_))
print("Selected Features are : %s" % (Feature.support_))
print("Feature Ranking is as follows: %s" % (Feature.ranking_))
The printed result was:
```

Number of Features: 3
Selected Features are : [True False False False False True True False]
Feature Ranking is as follows: [1 2 4 5 6 1 1 3]
Reading it: support_ is a True/False mask in the same order as the columns, and ranking_ gives 1 to every selected feature and larger numbers to features eliminated earlier. Matching positions to the column names (preg, plas, pres, skin, test, mass, pedi), the three selected features are **preg, mass, and pedi**. The ranking also tells us the order of elimination: the weakest features carry the highest rank numbers, so test (6) was eliminated first, then skin (5), pres (4), age (3), and plas (2). The warnings.filterwarnings('ignore') line just hides convergence warnings so the output is clean; it does not change the results.

This is also the point where Version 2 connects directly to Version 1: logistic regression is the model doing the judging, which satisfies the requirement that RFE needs a model that can report feature importance.

Embedded method: Ridge regression (L2)

```
from sklearn.linear_model import Ridge

ridge_reg = Ridge(alpha=1.0)
ridge_reg.fit(X, Y)

# A helper function for printing the coefficients
def print_coefs(coef, names=None, sort=False):
    if names == None:
        names = ["X%s" % x for x in range(len(coef))]
    lst = zip(coef, names)
    if sort:
        lst = sorted(lst, key=lambda x: -np.abs(x[0]))
    return " + ".join("%s * %s" % (round(coefs, 3), name)
```

```
for coefs, name in lst)
```

```
print("Ridge model:", print_coefs(ridge_reg.coef_))
```

feature influence rather than as a hard on/off selector. For a stricter version, you would use Lasso (L1).

Key takeaways from Version 2

- Feature selection methods can be organised into filter, wrapper, and embedded families, and knowing which family a technique belongs to tells you what to expect from it.
- Filter methods are quick and model-independent, but blind to feature interactions. Wrapper methods are more accurate but costly and risk overfitting. Embedded methods sit in the middle by doing selection during training.
- A method does not select anything by itself. You still have to choose k , the number of RFE features, or α , and those choices influence the result.
- The hands-on section is a clean pattern: load data, split X and Y , then apply one example from each family.

Combined Observations

How Version 1 and Version 2 connect

Version 1 gives the "what and why" in a compact, practical form, and Version 2 supplies the framework that organises it. Mapping the methods across makes this clear:

Technique	In Version 1	Where it sits in Version 2's framework
Variance Threshold	Basic filter	Unsupervised filter (does not use the target)
K Best Features	Filter by importance score	Filter method (SelectKBest with chi-squared)
RFE	Wrapper using a model's importances	Wrapper method (RFE with logistic regression)
Boruta	Wrapper-style, tree-model based, no manual threshold	Wrapper family (not shown in Version 2)

Regularisation/tree importances	Mentioned indirectly through "importance"	Embedded method (Ridge)
---------------------------------	---	-------------------------

So Version 1 is the tool list and Version 2 is the map. Version 2 also adds things Version 1 does not touch: information gain, Fisher's score, forward and backward selection, exhaustive search, the curse of dimensionality, and the advantages and disadvantages of each family. In the other direction, Version 1 has Boruta and Variance Threshold, which Version 2 does not cover.

There is also a difference in dataset and model. Version 1 uses the wine dataset with gradient boosting and F1 as the measuring stick, so it is concerned with proving performance. Version 2 uses the diabetes dataset with logistic regression and focuses on how each method is implemented and what its output looks like.

How this connects to earlier study in the programme

- **Python and pandas.** Earlier Python work covered data type conversion and collections like lists and tuples. That showed up again here: `df.values` converts a DataFrame into an array, slicing with `[:, 0:8]` is the same indexing logic as slicing a list, and `zip` and `sorted` in the Ridge helper function work on pairs of values much like tuples.
-

Useful, interesting and challenging parts

Most useful. The filter, wrapper, and embedded classification. Once I had that framework, individual techniques stopped feeling like a random list and became easier to place and compare. The baseline-first habit from Version 1 is also something I will reuse on any modelling task.

Most interesting. Boruta. Using shuffled copies of the real features to build a noise benchmark is a clever idea, and the 3-row example made it easy to see why a feature should beat the best shadow to be trusted.

Final thoughts

The single biggest lesson across both versions is that feature selection is a decision, not a button. The methods give evidence, but you still choose the thresholds, the number of features, and the model that does the judging, and you should always check the outcome against a baseline. If I had to sum it up in one line: start simple, measure against a baseline, and only pay for more expensive methods when they earn it.